

Vulnerability Assessment Report – Vuln Bank Web Application

Project / Task: OWASP Bootcamp Task 9

Repository: <https://github.com/Commando-X/vuln-bank>

Environment: Dockerized local setup (vuln-bank-web, vuln-bank-db)

Objective

The goal of this assessment was to identify and document security vulnerabilities in the intentionally vulnerable **Vuln Bank** web application, following OWASP testing principles. The task did not require remediation, only detection and documentation of vulnerabilities.

Scope

- Application: vuln-bank web app (front-end and back-end API)
- Database: PostgreSQL (Docker container)
- Environment: Local testing via Docker Desktop on Windows
- Tools used: Browser DevTools, docker exec, psql, manual inspection

Methodology

1. Cloned and deployed the vulnerable bank system using Docker.
2. Conducted manual black-box and gray-box testing to explore core functionalities (login, registration, password reset, transfers, file uploads, and admin panel).
3. Examined source code for insecure coding practices.
4. Verified findings via database inspection.
5. Documented each issue with description, evidence, and recommendation.

Findings Summary

#	Vulnerability	Severity	Category
1	SQL Injection in Login	Critical	Injection
2	Plaintext Password Storage & Weak Reset Flow	High	Cryptographic Failures

#	Vulnerability	Severity	Category
3	Broken Object Level Authorization (BOLA)	High	Access Control
4	Hardcoded Secret Key in Source Code	Low	Information Disclosure
5	Debug Mode Enabled in Production	Low	Security Misconfiguration

Detailed Findings

1. SQL Injection in Login

Severity: Critical

Location: app.py, login function (~line 285)

Description:

The login query concatenates user input directly into SQL without parameterization. Attackers can bypass authentication or extract sensitive data from the database.

Proof of Concept:

Username: ' OR '1'=1

Password: anything

Result: Login bypass successful.

Create Account

Registration successful! Proceed to login

'11'

..

Register

Recommendation:

Use parameterized queries or ORM methods to prevent SQL injection.

2. Plaintext Password Storage & Weak Reset Flow

Severity: High

Location: database.py (user table), app.py (reset logic lines 936–1034)

Description:

User passwords and reset PINs are stored in plaintext. The reset process does not require email verification and uses predictable 4-digit codes.

Proof of Concept (lab only):

```
docker exec -it vuln-bank-db-1 psql -U postgres -d vulnerable_bank \  
-c "SELECT username, password, reset_pin FROM users LIMIT 5;"
```

```
vulnerable_bank=# SELECT id, username, password, reset_pin, is_admin FROM users LIMIT 5;  
id | username | password | reset_pin | is_admin  
-----+-----+-----+-----+-----  
1  | admin   | admin123 |          | t  
2  | ann     | ann      | 804      | f  
3  | bob     | bob      |          | f  
(3 rows)
```

Recommendation:

Store only salted password hashes using bcrypt/argon2. Replace PINs with secure token-based resets.

3. Hardcoded Secret Key in Source Code

Severity: Low

Location: config.py or .env

Description:

Sensitive secret key is hardcoded in plain text in the source, risking token forgery if exposed.

```
# Hardcoded secret key (CWE-798)  
app.secret_key = "secret123"
```

Recommendation:

Store secrets in environment variables or secure key vaults.

4. Input Validation Flaw in Loan Request Form

Severity: Medium

Location: /loan-request or related endpoint

Description:

The system allows negative numbers in the loan amount field. This can be exploited to manipulate balances or trigger incorrect calculations.

Proof of Concept:

Submitted -5000 as loan amount, which was accepted successfully.

Your Loan Applications

Amount	Status
\$50000.00	pending
\$-1200	pending

Recommendation:

Implement **server-side input validation** to restrict loan amounts to positive integers only and sanitize numeric fields before processing.

5. Debug Mode Enabled in Production

Severity: Low

Location: app.py

Description:

The Flask application runs with `debug=True`. When enabled, Flask's debugger reveals sensitive details such as file paths, environment variables, and allows remote code execution if improperly exposed.

Proof of Concept:

Visiting an invalid route (e.g. `/random404`) shows a detailed debug traceback page.

```
PS C:\Users\LENOVO\Documents\vuln-bank> Select-String -Path app.py -Pattern "debug="
app.py:1943:      app.run(host='0.0.0.0', port=5000, debug=True)
```

Recommendation:

Disable debug mode in production by setting:

```
app.run(debug=False)
```

or use environment variables to toggle debug only during development.

Conclusion

The **Vuln Bank** web application demonstrates multiple critical and high-severity vulnerabilities, including **SQL Injection**, **Plaintext Password Storage**, and **Authorization Bypass**.

These issues provide an excellent opportunity to practice secure coding and OWASP Top 10 principles.

Future remediation should prioritize **input validation**, **proper cryptographic storage**, and **secure configuration management**.

Additional Screenshots

```
Windows PowerShell
[+] Running 5/5
  ✓ vuln-bank-web Built 0.0s
  ✓ Network vuln-bank_vuln_network Created 0.1s
  ✓ Volume vuln-bank_postgres_data Created 0.1s
  ✓ Container vuln-bank-db-1 Started 1.5s
  ✓ Container vuln-bank-web-1 Started 1.7s
PS C:\Users\LENOVO\Documents\vuln-bank> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED    STATUS    PORTS
09e213ac02d4   vuln-bank-web  "python app.py"          23 seconds ago Up 22 seconds 0.0.0.0:5000->5000/tcp, [::]:50
00->5000/tcp, 0.0.0.0:80->5000/tcp, [::]:80->5000/tcp
0eca27cb42b4   postgres:13    "docker-entrypoint.s..." 24 seconds ago Up 23 seconds 0.0.0.0:5432->5432/tcp, [::]:54
32->5432/tcp
PS C:\Users\LENOVO\Documents\vuln-bank> |
```

Ask Gordon BETA

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit BETA

Docker Hub

Docker Scout

Extensions

Containers [Give feedback](#)

Container CPU usage ⓘ
0.89% / 800% (8 CPUs available)

Container memory usage ⓘ
91.77MB / 3.61GB

Show charts

Search

Only show running containers

	Name	Container ID	Image	Port(s)	CPU (%)	Actions
☐	> vuln-bank	-	-	-	1.14%	

Walkthroughs

Multi-container applications

8 mins

\$ docker init

Containerize your application

3 mins

[View more in the Learning center](#)

